



Department of Computer Science and Engineering

ALGORITHMIC PROBLEM-SOLVING HACKATHON (APSH 2026)

PROJECT DOCUMENTATION

Date: 03-10-2026 | Venue: 3rd Floor, A & B Blocks

Field	Details
Project Title	Farmer Marketplace: A MERN-Based Platform Connecting Farmers and Customers with Greedy, Cost-Optimal Order Allocation
Team Name / Team No.	2
Year / Semester / Section	II Year / 3/ G
Algorithm Paradigm Used	☒ Greedy ☐ Divide & Conquer ☐ Dynamic Programming ☐ Backtracking ☐ Branch & Bound ☐ Graph
Programming Language / Tools	JavaScript (ES2022); Node.js, Express.js, React.js, MongoDB with Mongoose, JWT, bcrypt, Axios, Tailwind CSS, Postman, VS Code, Git

Team Members

S. No	Name of the Student	Roll No.
1	U. Vidyasri	25MVCSMR0621
2	S. Aishwarya	25MVCSMR0609
3	M. Hemanth	25MVCSMR0592
4	T. Ranganath	25MVCSMR0620

1. Problem Statement

Small and marginal farmers in India typically sell through a chain of commission agents and wholesalers. The farmer receives a low price, while the consumer pays a high one, and neither can see the other's real supply or demand. Customers also have no easy way to find fresh produce grown near them, and farmers have no simple tool to list stock and manage orders.

This project builds a **MERN-based Farmer Marketplace** in which farmers list their produce directly (product, quantity, price, location) and customers browse, search and order it. Three roles are supported: **Farmer**, **Customer** and **Admin**, each with role-based access to the system.

Beyond the standard CRUD operations, the project solves one specific algorithmic problem: **when a customer asks for Q kg of a produce item that is listed by several farmers at different prices and distances, which quantity should be bought from which farmer so that the total landed cost is minimum?** Formally, let listing i has price p_i (Rs/kg), available quantity q_i (kg) and distance d_i (km) from the customer. With a delivery rate r (Rs per kg per km), the landed unit cost is $c_i = p_i + r \times d_i$. We must choose x_i so that:

```
minimise    SUM over i of ( x_i × c_i )  
subject to  SUM over i of x_i = Q    and    0 ≤ x_i ≤ q_i    for every listing i
```

Aspect	Description
Inputs	Produce name, requested quantity Q (kg), customer location, the list of active listings (farmer, price, quantity, distance), delivery rate r (default Rs 0.5 per kg per km, configurable).
Outputs	An allocation plan (which farmer supplies how many kg), the minimum total landed cost, and the shortfall (if total supply is less than Q).
Constraints	Only approved, non-blocked farmers with quantity > 0 are considered. $0 \leq x_i \leq q_i$. Total allocated quantity never exceeds Q . Stock must never become negative, even when two customers order at the same time.
Scale	n = number of listings of one produce item (tens in a district, up to tens of thousands in a national deployment).

2. Objectives

1. **Build a complete MERN application** (MongoDB, Express.js, React.js, Node.js) that connects farmers and customers, with JWT authentication and role-based access for Farmer, Customer and Admin.
2. **Implement CRUD operations and REST APIs** for users, products (name, quantity, price, farmer, location) and orders, including search by name, category, price and location.
3. **Design a greedy order-allocation algorithm** that fulfils a customer's request at minimum landed cost (price + delivery cost), breaking ties by nearest farmer, and prove that it is optimal.
4. **Validate the solution** with test cases, check optimality against exhaustive search, and compare running time against a naive approach, presenting the results clearly.

3. Proposed Approach

3.1 System architecture

The system follows a three-tier architecture. The React client talks to the Express REST API over HTTPS using JSON; the API applies authentication, authorisation and business logic (including the greedy allocation service) and stores data in MongoDB through Mongoose.

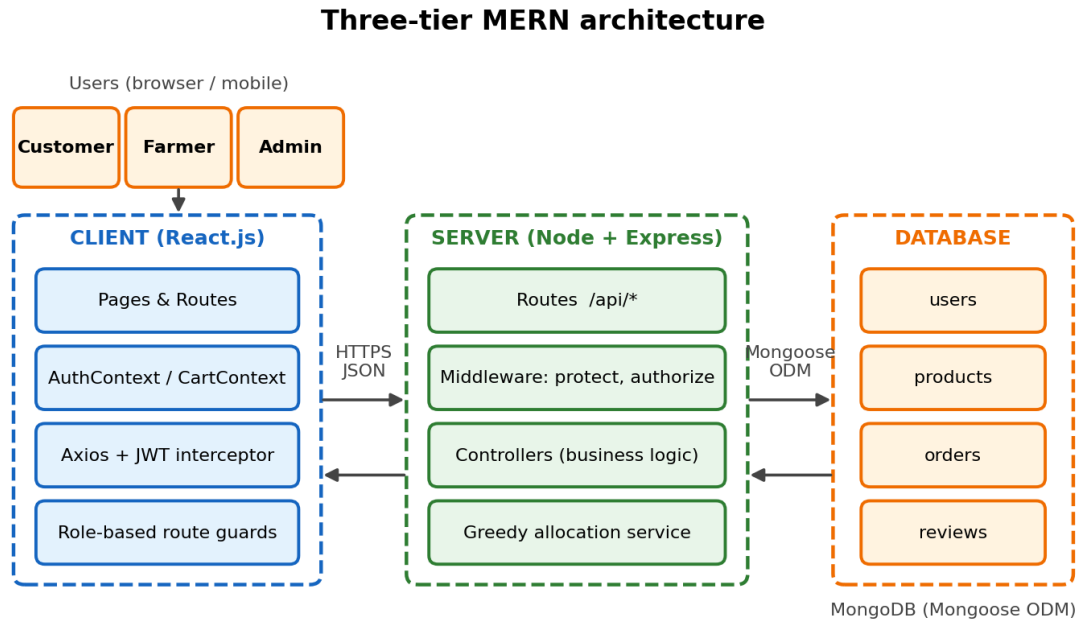


Fig. 1. Three-tier MERN architecture of the Farmer Marketplace

3.2 Algorithm paradigm chosen: Greedy

Every kilogram of produce costs the customer a fixed amount from a given farmer, and the cost is linear in the quantity taken. The problem therefore has the **greedy-choice property**: always buying the next kilogram from the cheapest listing that still has stock can never be worse than buying it elsewhere. The approach is:

1. Compute the landed unit cost $c_i = p_i + r \times d_i$ for every active listing.
2. Sort the listings by c_i (ascending), breaking ties by distance (ascending).
3. Walk down the sorted list, taking $\min(q_i, \text{remaining})$ from each listing until the order is filled or the listings run out.

Why it is optimal (exchange argument). Suppose an optimal plan buys some quantity from listing j while a cheaper listing i ($c_i < c_j$) still has unused stock. Moving a small quantity δ from j to i keeps the total quantity unchanged and changes the cost by $\delta(c_i - c_j) < 0$, which is a strict improvement. This contradicts optimality, so in an optimal plan every cheaper listing is exhausted before a costlier one is used. This is exactly the plan the greedy algorithm builds. Because capacities and Q are whole kilograms, the greedy plan is also integral.

3.3 Alternatives considered

Approach	Idea	Why it was not chosen
Cheapest price only (naive greedy)	Sort by farmer price p_i and ignore distance.	Ignores delivery cost. In our test data it costs 11.1% more than the proposed method (Section 7.3).
Brute force / Backtracking	Try every split of Q across the n listings.	Exponential in n . Used only on small cases to verify that greedy is optimal.
Dynamic Programming (bounded knapsack)	$dp[i][q]$ = minimum cost to buy q kg from the first i listings.	Correct, but $O(n \times Q)$ time and memory. Needed only if costs are non-linear, for example minimum lot sizes or volume discounts. Listed as future scope.
Branch & Bound	Prune partial plans using a lower bound on cost.	Same reason as DP: powerful for non-linear constraints, but unnecessary for linear costs.

Approach	Idea	Why it was not chosen
Graph (Dijkstra / TSP)	Optimise delivery routes between farms and customers.	Solves a different problem (route planning). Distances here come from the Haversine formula; route optimisation is future scope.

3.4 Buying modes

A customer can (a) **buy directly** from a chosen farmer's listing, or (b) use **Best Price Match**, where the customer enters a produce name and quantity and the greedy service builds the cheapest multi-farmer plan. Listings with the same produce name and category are treated as interchangeable.

4. Algorithm / Pseudocode

Algorithm 1: Greedy cost-optimal allocation

```

ALGORITHM GreedyAllocate(L[1..n], Q, r)
Input : L[i] = (farmer, price, qty, dist);  Q = quantity requested;  r = delivery rate
Output: plan (farmer, kg, unit cost),  total landed cost,  shortfall

1  candidates <- { L[i] : L[i].qty > 0 }
2  for each c in candidates do  c.cost <- c.price + r * c.dist
3  SORT candidates by (cost ascending, dist ascending)
4  need <- Q ;  total <- 0 ;  plan <- empty list
5  for each c in candidates, in sorted order do
6      if need = 0 then break
7      take <- min(c.qty, need)
8      append (c.farmer, take, c.cost) to plan
9      total <- total + take * c.cost
10     need <- need - take
11  return (plan, total, shortfall = need)

```

Algorithm 2: Order placement with safe stock update

```

ALGORITHM PlaceOrder(customer, produce, Q, address)
1  listings <- approved listings of produce, with Haversine distance to customer
2  (plan, total, shortfall) <- GreedyAllocate(listings, Q, r)
3  if shortfall > 0 then return "insufficient stock"
4  for each (farmer, take) in plan do
5      ok <- atomic: quantity <- quantity - take, only if quantity >= take
6      if not ok: undo earlier decrements, retry from step 1 (max 3 times)
7  group the plan lines by farmer
8  create Order(customer, items, totalAmount = total, status = "pending", address)
9  return Order

```

Distance (Haversine). For latitudes ϕ_1, ϕ_2 and longitudes λ_1, λ_2 in radians, with $R = 6371$ km: $a = \sin^2((\phi_2 - \phi_1)/2) + \cos \phi_1 \cdot \cos \phi_2 \cdot \sin^2((\lambda_2 - \lambda_1)/2)$ and $d = 2R \cdot \text{atan2}(\sqrt{a}, \sqrt{1 - a})$.

The atomic update in Algorithm 2 (a single MongoDB `findOneAndUpdate` with the condition `quantity >= take`) guarantees that two simultaneous orders can never drive stock below zero; the loser of the race simply retries with fresh data.

Flowchart

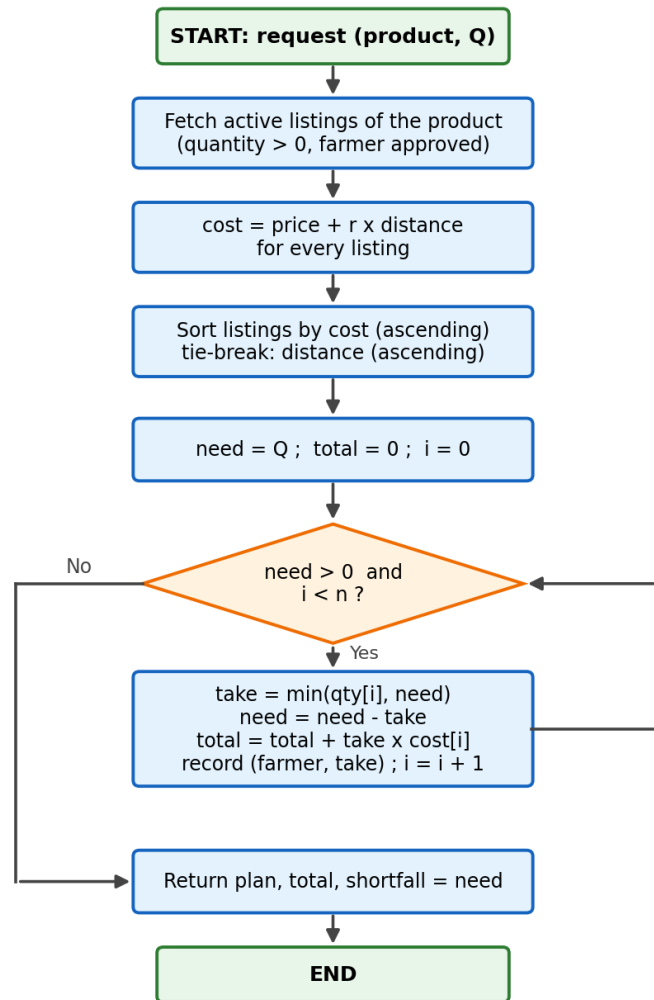


Fig. 2. Flowchart of Algorithm 1 (GreedyAllocate)

5. Complexity Analysis

Let n be the number of active listings of the requested produce and k the number of farmers actually used in the plan ($k \leq n$).

Complexity	Best Case	Average Case	Worst Case
Time	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Space	$O(n)$	$O(n)$	$O(n)$

- **Cost computation (step 2):** one pass over the listings, $O(n)$.
- **Sorting (step 3):** comparison sort, $O(n \log n)$. This dominates, so every case is $O(n \log n)$, even when a single farmer can fill the order.
- **Greedy scan (steps 5–10):** each listing is visited at most once, $O(n)$; it stops early once $\text{need} = 0$.
- **Space:** one array of n cost-annotated listings plus a plan of at most k entries, so $O(n)$.
- **Order placement:** Algorithm 2 adds k atomic database updates, giving $O(n \log n + k)$ overall, with $k \leq n$. Fetching listings uses an index on produce name and quantity.
- **Possible refinement:** replacing the full sort by a min-heap (build in $O(n)$, extract k times in $O(\log n)$ each) gives $O(n + k \log n)$, which is faster when k is much smaller than n .

6. Implementation

6.1 Modules

Layer	Module / files	Responsibility
Server	config/db.js	Connects to MongoDB through Mongoose using an environment variable.
Server	models/User.js, Product.js, Order.js	Mongoose schemas with validation and indexes.
Server	middleware/auth.js, role.js, error.js	protect verifies the JWT, authorize(...roles) enforces the role, a central handler formats errors.
Server	controllers/*, routes/*	Auth, product CRUD, orders and admin operations exposed as REST endpoints.
Server	services/allocation.js, utils/haversine.js	The greedy allocation (Algorithm 1) and the distance calculation.
Client	pages/*	Home, Login/Register, Product List, Product Detail, Cart, Checkout, My Orders, Farmer Dashboard, Add/Edit Product, Farmer Orders, Admin Panel.
Client	context/AuthContext, CartContext	Global login state, role and cart.
Client	api/axios.js, components/ProtectedRoute	Axios instance that attaches the JWT; route guards that redirect users who lack the required role.

6.2 Data structures and database design

- **Arrays of listing objects** hold the candidate listings; they are sorted in place by (cost, distance).
- **Plan list** (array of farmer, kg, unit cost) is the output of the algorithm.
- **Hash map** keyed by farmer id groups plan lines into one order item per farmer.
- **MongoDB documents** with references (farmer, customer, product) and the indexes listed below.

Collection	Main fields	Indexes
users	name, email, password (bcrypt hash), role (farmer customer admin), phone, location (district, state, coordinates), isApproved, isBlocked	email (unique); 2dsphere on location
products	name, category, price, quantity, unit, description, imageUrl, location, farmer (ref users), createdAt	text on name; (category, price); 2dsphere on location
orders	customer (ref), items [product, farmer, qty, price], totalAmount, status (pending accepted shipped delivered cancelled), address, paymentMode (COD), createdAt	(customer, createdAt); (items.farmer, status)
reviews (optional)	product (ref), customer (ref), rating (1–5), comment	(product)

6.3 REST API

Method	Endpoint	Access	Purpose
POST	/api/auth/register, /api/auth/login	Public	Create an account; receive a JWT.
GET	/api/auth/me	Any user	Current user profile.
GET	/api/products?search=&category=&location=&page=	Public	List and filter products with pagination.
GET	/api/products/:id	Public	Product detail.
POST	/api/products	Farmer	Add a product.
PUT / DELETE	/api/products/:id	Owner (DELETE also Admin)	Update or remove a product.
POST	/api/orders/quote	Customer	Preview the Best Price Match plan (Algorithm 1).

Method	Endpoint	Access	Purpose
POST	/api/orders	Customer	Place an order (Algorithm 2); stock decreases.
GET	/api/orders/my	Customer	Order history.
GET	/api/orders/farmer	Farmer	Orders containing the farmer's products.
PUT	/api/orders/:id/status	Farmer	Accept, ship or cancel an order.
GET / PUT / DELETE	/api/admin/users, /users/:id/approve, /products/:id, /stats	Admin	Approve or block farmers, remove listings, view totals.

6.4 Security and validation

- Passwords are hashed with bcrypt; sessions use signed JWTs sent in the Authorization: Bearer header.
- Role checks are done on the server (authorize), not only in the UI. A farmer can modify only products whose farmer equals the logged-in user.
- Inputs are validated (type, range, required fields) before reaching the database; secrets live in environment variables.

6.5 Core service (tested)

services/allocation.js

```
function allocate(listings, Q, r = 0.5) {
  const ranked = listings
    .filter(l => l.qty > 0)
    .map(l => ({ ...l, cost: l.price + r * l.dist }))
    .sort((a, b) => a.cost - b.cost || a.dist - b.dist);
  let need = Q, total = 0;
  const plan = [];
  for (const l of ranked) {
    if (need <= 0) break;
    const take = Math.min(l.qty, need);
    plan.push({ farmer: l.farmer, qty: take, unit: +l.cost.toFixed(2) });
    total += take * l.cost;
    need -= take;
  }
  return { plan, total: +total.toFixed(2), shortfall: Math.max(need, 0) };
}
```

6.6 Tools and platform

Node.js (LTS) with Express.js for the API; React.js with React Router for the interface; MongoDB (local or Atlas) with Mongoose; Axios; Tailwind CSS; Multer for image upload; Postman for API testing; Git and GitHub for version control; VS Code as the editor. Planned deployment: Render or Railway (API), Vercel or Netlify (client), MongoDB Atlas (database).

7. Results and Test Cases

7.1 Algorithm test cases

The allocation service was executed in Node.js on the following sample data, with $r = \text{Rs } 0.5$ per kg per km. The last column is the landed cost per kg, $c_i = p_i + r \times d_i$. Note that Lakshmi has the lowest price but the highest landed cost because she is the farthest.

Farmer	Price (Rs/kg)	Stock (kg)	Distance (km)	Landed cost (Rs/kg)
Ravi	30	40	12	36.00
Sita	28	25	30	43.00
Kiran	32	50	5	34.50
Lakshmi	26	15	60	56.00

ID	Scenario	Q (kg)	Allocation produced	Total (Rs)	Short (kg)	Result
TC1	Small order, one farmer can fulfil it	30	Kiran 30 kg @ 34.50	1,035.00	0	Pass
TC2	Order spans three farmers	100	Kiran 50 kg @ 34.50; Ravi 40 kg @ 36.00; Sita 10 kg @ 43.00	3,595.00	0	Pass
TC3	Order equals total supply (all four farmers)	130	Kiran 50 kg @ 34.50; Ravi 40 kg @ 36.00; Sita 25 kg @ 43.00; Lakshmi 15 kg @ 56.00	5,080.00	0	Pass
TC4	Zero quantity requested	0	(none)	0.00	0	Pass
TC5	Equal landed cost: tie broken by distance	15	B 10 kg @ 25.00; A 5 kg @ 25.00 (listings: A: Rs 20, 10 kg, 10 km; B: Rs 25, 10 kg, 0 km)	375.00	0	Pass
TC6	Only listing is out of stock	5	(none) (single listing with 0 kg stock)	0.00	5	Pass
TC7	Demand exceeds total supply	140	Kiran 50 kg @ 34.50; Ravi 40 kg @ 36.00; Sita 25 kg @ 43.00; Lakshmi 15 kg @ 56.00	5,080.00	10	Pass

Hand check of TC2: $Kiran\ 50 \times 34.5 = 1,725$; $Ravi\ 40 \times 36 = 1,440$; $Sita\ 10 \times 43 = 430$; total = Rs 3,595, which matches the program output. All seven cases produced the expected result.

7.2 Optimality check

To confirm the proof experimentally, the greedy result was compared with an exhaustive search on 300 randomly generated instances (4 listings each, random price Rs 10–49, stock 1–6 kg, distance 0–49 km, random Q within total supply). **The greedy total equalled the exhaustive optimum in all 300 of 300 instances.**

7.3 Comparison with naive approaches

(a) Cheapest price only. For TC2 ($Q = 100$ kg) a strategy that ignores distance buys Lakshmi 15 kg, Sita 25 kg, Ravi 40 kg and Kiran 20 kg, which costs $840 + 1,075 + 1,440 + 690 = \text{Rs } 4,045$. The proposed method costs Rs 3,595, a saving of **Rs 450 (11.1%)** for the customer.

(b) Naive selection loop. A naive method that repeatedly scans all listings for the cheapest one with stock takes $O(n \times k)$ time. Both methods were run in Node.js on random data with $Q = 2.5\ n$ (single run each; absolute times depend on the machine).

Listings (n)	Naive scan (ms)	Greedy: sort + one pass (ms)	Speed-up
1,000	12.2	2.3	5.3×
10,000	312.1	19.2	16.3×
50,000	7,634.9	126.2	60.5×

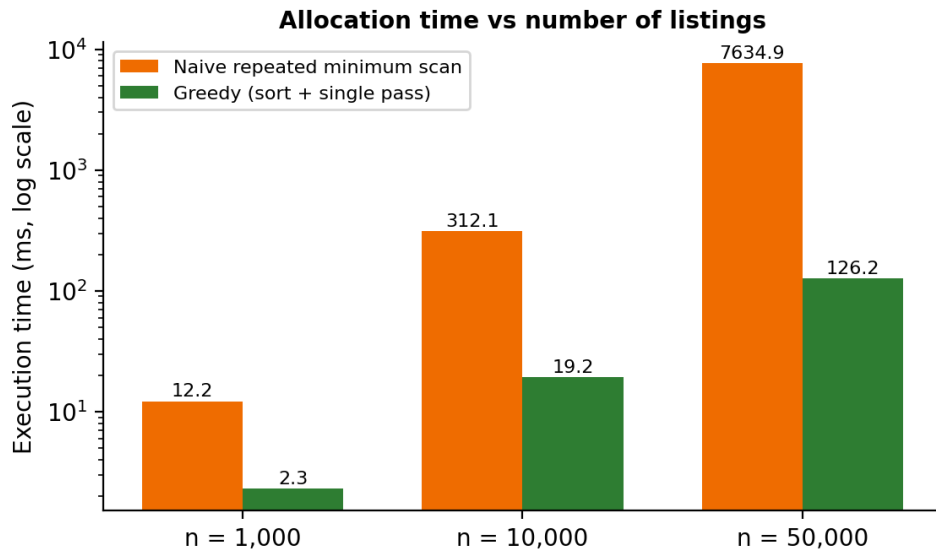


Fig. 3. Allocation time versus number of listings (log scale)

7.4 Functional test cases for the application

These cases are to be executed against the running application (use Postman for the API checks) and the Status column filled in.

ID	Test	Expected result	Status
FT1	Register as a farmer	Account created with role farmer and isApproved = false.	☐ Pass ☐ Fail
FT2	Login with correct and wrong password	JWT returned for correct credentials; 401 for wrong ones.	☐ Pass ☐ Fail
FT3	Customer calls POST /api/products	403 Forbidden.	☐ Pass ☐ Fail
FT4	Farmer edits another farmer's product	403 Forbidden; product unchanged.	☐ Pass ☐ Fail
FT5	Order more than the available stock	Rejected with a clear message; stock unchanged.	☐ Pass ☐ Fail
FT6	Two customers order the last 10 kg at once	One order succeeds, the other fails; stock is never negative.	☐ Pass ☐ Fail
FT7	Farmer marks an order as shipped	Customer sees status "shipped" in My Orders.	☐ Pass ☐ Fail
FT8	Admin blocks a farmer	Farmer's products disappear from search results.	☐ Pass ☐ Fail

7.5 Application screenshots

[Insert screenshot 1: Home / product list with filters]

Fig. 4. Home / product list with filters

[Insert screenshot 2: Farmer dashboard and Add Product form]

Fig. 5. Farmer dashboard and Add Product form

[Insert screenshot 3: Cart, Best Price Match plan and checkout]

Fig. 6. Cart, Best Price Match plan and checkout

[Insert screenshot 4: Admin panel with statistics]

Fig. 7. Admin panel with statistics

8. Conclusion and Future Scope

The Farmer Marketplace gives farmers a direct channel to customers and gives customers a transparent, searchable view of local produce. The MERN design covers authentication, role-based access, CRUD operations and REST APIs for products, orders and users. The greedy allocation algorithm picks the cheapest combination of farmers for an order; it was proven optimal by an exchange argument, matched exhaustive search on all 300 random instances, saved 11.1% over a price-only strategy on the sample data, and ran up to 60 times faster than a naive scan at 50,000 listings.

Limitations. The greedy method assumes linear costs and freely divisible stock. Minimum order sizes, volume discounts or per-farmer delivery fees would break this assumption.

Future scope:

- Minimum-lot and discount constraints solved with dynamic programming or branch and bound.
- Delivery route optimisation for multi-farmer orders using graph algorithms.
- Online payments (UPI / cards), SMS or WhatsApp order alerts, and Telugu / Hindi language support.
- Nearby search with MongoDB 2dsphere queries, ratings and reviews, and price suggestions from market data.
- Progressive Web App or mobile app, and cloud deployment with automated tests.

9. References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [2] J. Kleinberg and É. Tardos, *Algorithm Design*. Boston, MA, USA: Addison-Wesley, 2005.
- [3] S. Dasgupta, C. Papadimitriou, and U. Vazirani, *Algorithms*. New York, NY, USA: McGraw-Hill, 2008.
- [4] R. W. Sinnott, "Virtues of the Haversine," *Sky and Telescope*, vol. 68, no. 2, p. 159, 1984.
- [5] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF, RFC 7519, May 2015.
- [6] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annu. Tech. Conf. (FREENIX Track)*, Monterey, CA, USA, 1999, pp. 81–91.
- [7] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Univ. California, Irvine, CA, USA, 2000.
- [8] MongoDB Inc., "MongoDB Manual." [Online]. Available: <https://www.mongodb.com/docs/manual/>
- [9] Mongoose, "Mongoose documentation." [Online]. Available: <https://mongoosejs.com/docs/>
- [10] Express, "Express – Node.js web application framework." [Online]. Available: <https://expressjs.com/>
- [11] Meta Platforms, Inc., "React documentation." [Online]. Available: <https://react.dev/>
- [12] OpenJS Foundation, "Node.js documentation." [Online]. Available: <https://nodejs.org/docs/>